Instituto Infnet

Quem escolhe, mede; quem pesa, carrega uma vez

Projeto de Bloco: Inteligência Artificial Aplicada

Aulas 15–16 · Etapa 8 (versão compacta) · O modelo de linguagem — TP4, itens 1 e 2

Prof. Renan Santos

Onde paramos (Aula 14)

A rota do TP4 tem contrato e sabe falhar — falta o modelo.

- **POST /processar_texto: PedidoAnalise → ResultadoAnalise**
  o contrato da Aula 13.
- **O 503 quando o motor não carrega**
  a Aula 14 — e o resto da API no ar.
- **O motor ainda é provisório**
  palavras-chave: 13 de 30 certos, 17 sem resposta.

▸ **Hoje: QUAL modelo — e como ele entra na API sem derrubar nada.**

🎬 'Põe uma IA nisso' — e depois?

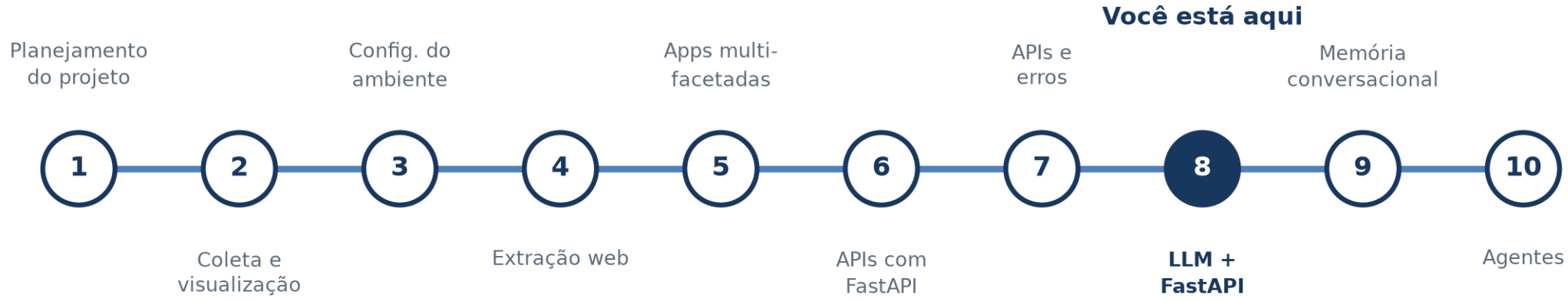
Duas perguntas, uma etapa.

- **Seis de dez relatos voltaram 'sem pista'**
  'Põe uma IA nisso.' Três colegas escolheram: o mais baixado, 'um LLM de verdade', um multilíngue — e os três tropeçaram.
- **O escolhido entrou na API — e a semana foi longa**
  cada análise levava 10 s; o painel congelava; a API ficou muda na subida; uma análise deu 500.

▸ **Escolher medindo; integrar sem que o peso do modelo derrube o resto.**

Onde estamos no Projeto de Bloco

Etapa 8 inteira, hoje: escolher (4.3) e integrar (4.4). Depois, o TP4 está completo.



Objetivos da aula

Do Hub à rota — cada passo medido.

- **Escolher com critérios declarados**
  a ficha (acessibilidade), o custo na sua máquina, o banco de provas.
- **Justificar — com o plano B**
  o item 1 do TP4: docs/escolha_do_modelo.md.
- **Carregar uma vez, sem travar**
  o lifespan; a rota def.
- **Garantir o local e o contrato**
  baixar antes; local_files_only; 503; pontuar as categorias.

O que esta aula entrega ao TP4

Os dois itens que faltavam.

- **Item 1: "Justifique a escolha do modelo… desempenho, custo computacional local e acessibilidade"**
  a ficha, a bancada, o documento.
- **Item 2: "integre-o ao seu backend FastAPI, sem necessidade de conexão com a nuvem"**
  o modelo atrás de POST /processar_texto; o download à parte.
- **Itens 3 e 4 (Aulas 13–14)**
  o contrato e os erros — agora com o modelo de verdade.

▸ **Depois de hoje: os quatro itens do TP4 no projeto.**

1 · Escolher: quem escolhe, mede

Passo 1 · um modelo do Hub em três linhas

passo01_pipeline.py

```
from transformers import pipeline

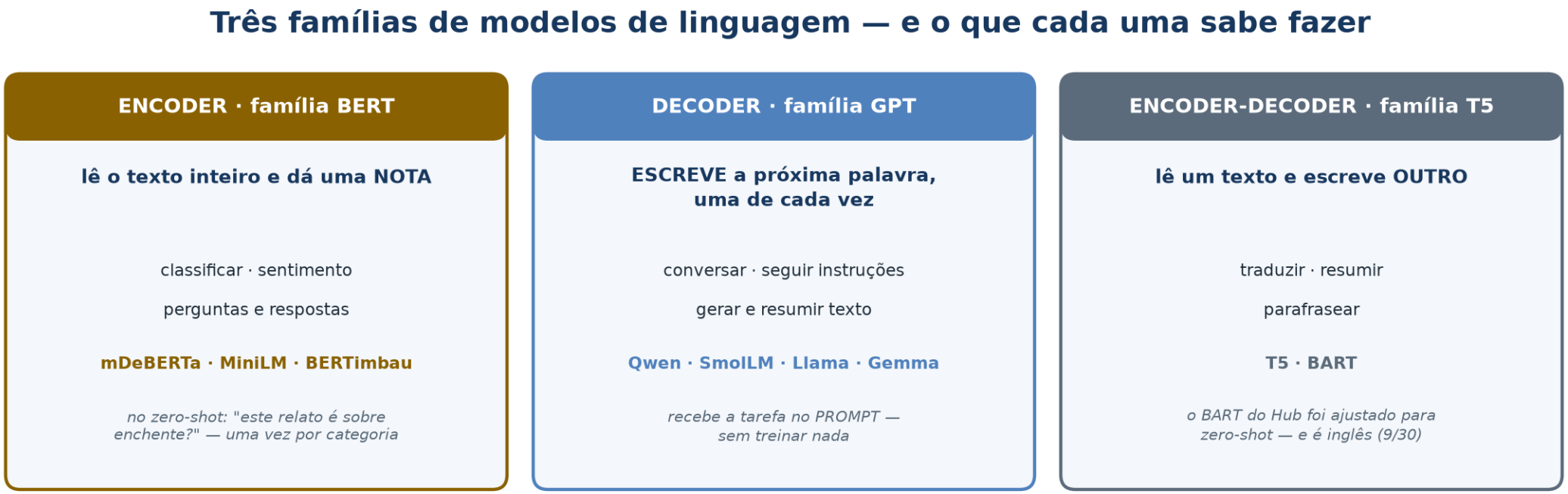
MODELO = "MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7"
CATEGORIAS = ["queimada", "desmatamento", "enchente", "seca", "poluição", "lixo"]

classificador = pipeline("zero-shot-classification", model=MODELO)   # carregar
r = classificador(ARROIO, candidate_labels=CATEGORIAS,                # perguntar
                  hypothesis_template="Este relato é sobre {}.")

# enchente 0.80 · seca 0.10   ← o arroio; as regras da Aula 13: null
# analisado em ~5 s           ← guarde este número
```

Três famílias — o 'GPT, BERT, T5' do TP4

Classificar dá para fazer com as três. Qual acerta mais no SEU dado, só medindo. [Alto, p36]



TP4: "o tipo de tarefa para a qual o modelo foi treinado (ex.: GPT, BERT, T5)" — [Alto, p36] · [Parandeh, p96–97]

🐛 Passo 2 · a ficha, antes de baixar

[ASK antes] Destes oito, qual vai bem em português?

- **facebook/bart-large-mnli — o zero-shot MAIS BAIXADO (2,4 mi/mês)**
  não declara língua: é inglês.
- **MiniLM 'multilingual'**
  15 línguas no cartão — nenhuma é o português.
- **google/gemma-3-1b-it**
  gated: OSError 'You are trying to access a gated repo… 401'.
- **BERTimbau (português!) · Qwen2.5-3B**
  tarefa fill-mask (não classifica) · licença 'other'.
- **Qwen2.5-1.5B-Instruct**
  o cartão diz só 'en' — e é o que melhor entende os relatos.

▸ **O cartão é pista; o banco de provas é veredito.**

🐛 Passo 3 · float16 × float32

passo03_custo.py

```
DTYPE = "auto"        # "auto" = o tipo em que o modelo foi SALVO · troque para "float32"
pipe = pipeline("zero-shot-classification", model=MODELO,
                dtype="auto" if DTYPE == "auto" else getattr(torch, DTYPE))
memoria = sum(p.numel() * p.element_size() for p in pipe.model.parameters())

# auto     → float16 · 279 mi × 2 bytes =   558 MB · 4,74 s por relato
# float32  → float32 · 279 mi × 4 bytes = 1.115 MB · 0,61 s por relato
```

Custo é memória E tempo

O tipo dos números muda os dois — e a CPU tem preferências.

- **float16 e bfloat16: metade da memória**
  pensados para placas de vídeo; numa CPU sem suporte nativo, lentos.
- **mDeBERTa: 558 MB e 4,74 s × 1,1 GB e 0,61 s**
  o dobro de memória comprou 8 vezes a velocidade.
- **Qwen2.5-1.5B: 3,1 GB e 12,9 s × 6,2 GB e 2,8 s**
  o mesmo acerto (27 de 30) — nesta CPU.
- **A conta de padaria**
  parâmetros × bytes. 1,5 bilhão × 4 ≈ 6 GB. O Hub sabe antes do download.

▸ **Os segundos são da SUA máquina. Meça — e anote a máquina.**

O banco de provas

'a top performer in one specific benchmark' [Alto, p82] — o benchmark que importa é o nosso.

- **30 relatos fictícios, 5 por categoria, rotulados à mão**
  data/analisador/banco_de_provas.csv
- **Metade com as palavras das regras, metade sem**
  como o arroio — para medir o que o modelo acrescenta.
- **Os limites, ditos agora**
  30 é pouco (um erro = 3,3 pontos); escrevemos nós; um rótulo por relato.

▸ **Rotule ANTES de rodar qualquer modelo.**

Passo 4 · errar × calar

[ASK] Qual dos dois é melhor?

- **regras: 13 de 30 certos · 17 sem resposta · 0 errados**
  quando não sabem, calam.
- **mDeBERTa: 18 de 30 certos · 0 sem resposta · 12 errados**
  sempre apostam — 'fumaça densa' → poluição.
- **Depende de quem lê a resposta**
  um alarme conservador × uma triagem que precisa de uma categoria.

▸ **Errar e calar não são a mesma coisa — conte os dois.**

🐛 Passo 5 · o LLM que escreve

```
passo05_gerador.py — Qwen2.5-0.5B

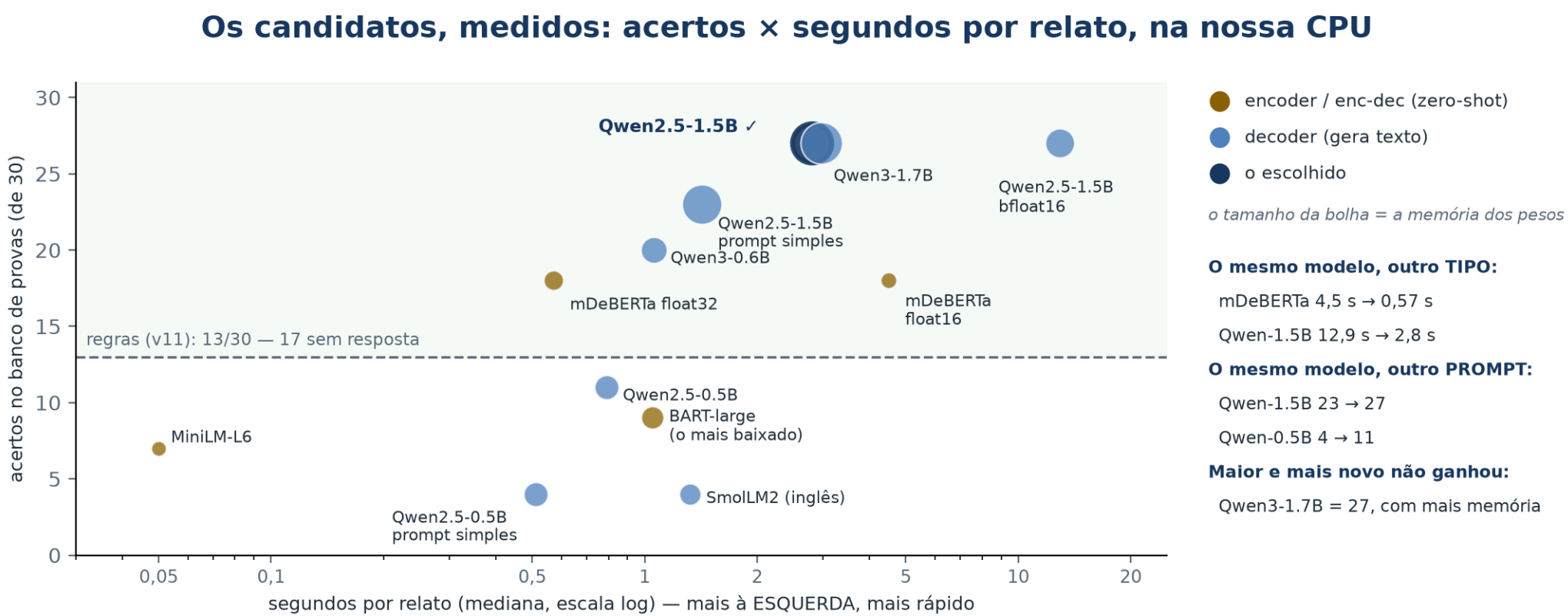
PROMPT = "simples"      # só a lista de categorias

✗ # 2 seca          o modelo escreveu: 'Desmatamento'
✗ # 4 enchente      o modelo escreveu: 'Desmatamento'
✗ #17 enchente      o modelo escreveu: 'Chuveiro'
✗ #25 poluicao      o modelo escreveu: 'chama-vaquinha'
Qwen/Qwen2.5-0.5B-Instruct · prompt simples:    4 de 30 certos

PROMPT = "definicoes"   # o que CADA categoria quer dizer
Qwen/Qwen2.5-0.5B-Instruct · prompt definicoes: 11 de 30 certos
```

O mapa dos candidatos

13 arranjos, a mesma bancada, a mesma CPU. A bolha é a memória.



Fonte: src/bancada.py → data/analisador/avaliacao.json (27/10/2026). Windows 11, 12 núcleos lógicos, só CPU. Noutra máquina, os segundos mudam — meça.

A decisão — e o que pesou contra

Qwen/Qwen2.5-1.5B-Instruct · float32 · CPU · prompt com definições

- **Desempenho**
  27 de 30 — as regras, 13; o melhor zero-shot, 18.
- **Custo computacional local**
  3,1 GB de download · 6,2 GB de memória · 2,8 s por relato · ~16 s para carregar.
- **Acessibilidade local**
  Apache 2.0 · não é gated · torch de CPU · offline depois de baixado.
- **Contra: a memória — plano B medido**
  bfloat16 (3,1 GB, 27, 12,9 s) ou Qwen3-0.6B (20, 1,1 s, 2,4 GB).

▸ **Onde ele erra: 3 relatos — um deles as regras acertam. Isso também vai no documento.**

2 · Integrar: quem pesa, carrega uma vez

🐛 Passo 1 · carregar a cada pedido

passo01_por_requisicao.py

```
@app.post("/processar_texto")
def processar_texto(pedido: Pedido):
    motor = carregar()              # 🐛 a cada pedido: do disco para a memória
    return classificar(pedido.texto, motor)

# python cronometro.py analisar 3
#   POST → 200 em 6,1 s      [motor] carregado em 5,2 s
#   POST → 200 em 2,1 s      [motor] carregado em 1,6 s
#   POST → 200 em 2,1 s      [motor] carregado em 1,5 s
```

Passo 2 · carregar uma vez: o lifespan

passo02_lifespan.py

```
MOTORES = {}                                  # o modelo carregado mora aqui

@asynccontextmanager
async def lifespan(_: FastAPI):
    MOTORES["analise"] = carregar()           # quando a API SOBE — uma vez
    yield
    MOTORES.clear()                           # quando a API DESCE

app = FastAPI(lifespan=lifespan)

# [motor] carregado em 3,7 s  →  Application startup complete
# cada POST: 0,4 s
```

🐛 Passo 3 · async def × def

passo03_async_trava.py

```
TRAVAR = True

if TRAVAR:
    @app.post("/processar_texto")
    async def processar_texto(pedido: Pedido):     # no laço principal
        return classificar(pedido.texto, MOTORES["analise"])
else:
    @app.post("/processar_texto")
    def processar_texto(pedido: Pedido):           # numa thread à parte
        return classificar(pedido.texto, MOTORES["analise"])

# python cronometro.py junto   (relato de 1.899 caracteres + GET /saude aos 0,5 s)
#   async def → /saude esperou 1,64 s      def → /saude em 0,00 s
```

A regra

Pensar na CPU é bloquear — não há await nenhum ali.

- **A rota que chama o modelo local é def**
  na v14: def processar_texto(…), com o comentário dizendo por quê.
- **async def é para quem ESPERA algo de fora**
  await numa chamada de rede — não para quem calcula.
- **Dois pedidos ao mesmo tempo dividem a CPU**
  medido: 4,6 s cada (sozinho, ~2,5 s) — e /desmatamento em 0,05 s.

▸ **Até hoje, toda rota do curso foi def. Para o modelo, continua sendo.**

🐛 Passo 4 · o modelo que não carrega

[ASK antes] Com o nome do modelo errado, quais rotas sobem?

- **PROTEGER = False (sem try)**
  OSError … → ERROR: Application startup failed. Exiting. — nem o /saude responde.
- **PROTEGER = True (com try)**
  ERROR: Modelo NÃO carregado — OSError… · Application startup complete.
- **GET /saude → {'ok': true, 'modelo_carregado': false}**
  POST /processar_texto → 503, com a frase.

▸ **Uma exceção no lifespan derruba a subida inteira. Falhe em pedaços (Aula 14).**

Baixar antes, ler offline

Sem o modelo no disco — medido, com o cache vazio:

- **Carregando pelo nome, COM internet**
  a subida baixa 3,1 GB: 71 s sem responder a nada.
- **Carregando pelo nome, SEM internet**
  5 tentativas, espera crescente: OSError depois de 82,5 s.
- **Com local_files_only=True**
  OSError em 0,4 s → 503. O modelo ou está no disco, ou não está.
- **O download vira um passo à parte**
  python -m src.baixar_modelo — como a coleta das Aulas 7 e 10.

▸ **TP4: 'sem depender de serviços externos'. O livro não trata disso (zero ocorrências).**

🐛 Passo 5 · 'Chuveiro' não é categoria

passo05_contrato.py — PONTUAR = False

```
class Resultado(BaseModel):
    categoria: Literal["queimada", "desmatamento", "enchente", "seca", "poluicao", "lixo"]

resposta = escrever(pedido.texto, motor)        # o modelo ESCREVE
return {"categoria": resposta.lower(), "confianca": None}

# 'Choveu sem parar desde sexta e a água já cobre o campo…'  →  500
# ResponseValidationError: {'type': 'literal_error',
#   'loc': ('response', 'categoria'), 'input': 'chuveiro'}
```

Pontuar as seis: o modelo não escreve

src/analisador_llm.py (v14)

```
entrada = tokenizador.apply_chat_template(mensagens, add_generation_prompt=True,
                                          return_tensors="pt", return_dict=True)
with torch.no_grad():
    saida = modelo(**entrada)
notas = saida.logits[0, -1, tokens_das_categorias]   # a PRÓXIMA palavra, só as seis
probabilidades = torch.softmax(notas, dim=0)         # somam 1

# o 1º token de cada categoria: que · des · ench · se · pol · lix
# bancada (1.5B): 27 de 30 — o mesmo de quando escrevia — em 2,2 s
```

A confiança não é chance de acerto

O modelo errou o nº 2 com confiança 1,00.

- **'O nível do rio baixou tanto que o barco não chega ao porto' — seca**
  o modelo: enchente, 1,00. (As regras acertam esse.)
- **25 dos 27 acertos vêm com ≥ 0,94**
  e os três erros vieram com 1,00, 0,81 e 0,96.
- **A confiança alta não separa o certo do errado**
  por isso a página avisa — e a categoria do relato é a de quem o enviou.

▸ **Diga ao usuário quem analisou — e o que o número NÃO é.**

O TP4 completo

O enunciado, item por item — todos no projeto.

- ✅ **1 — a escolha e a justificativa**
  docs/escolha_do_modelo.md (Aula 15).
- ✅ **2 — o modelo local atrás de POST /processar_texto**
  analisador_llm · baixar_modelo · lifespan · def (hoje).
- ✅ **3 — response models, respostas validadas**
  ResultadoAnalise (Aula 13) — a categoria sempre da lista.
- ✅ **4 — o 503 do modelo, a entrada, o timeout, o feedback**
  Aula 14 — e hoje o 503 é do modelo.

▸ **Entrega: 16–21/11. No README: a memória e o tempo que o modelo pede, e como baixá-lo.**

A ponte para a Etapa 9

O modelo é de CONVERSA — e já está carregado na API.

- **Hoje ele classifica: uma pergunta, uma resposta**
  sem lembrar nada.
- **Etapa 9: memória conversacional**
  o que foi dito antes entra no próximo prompt.

> **O peso já está no lugar. Falta a memória.**

Anti-padrões da Etapa 8

Na escolha e na integração — todos parecem razoáveis até a primeira medida.

- **Escolher pelos downloads, pelo tamanho, pelo nome 'LLM'**
  o BART (9/30); o Qwen3-1.7B (= 27, mais memória).
- **O dtype no automático · esconder onde o modelo erra**
  4,5 s onde cabiam 0,57; o erro de 100%.
- **Carregar na rota · async def · lifespan sem try · baixar na subida**
  cada pedido paga a carga; a API trava; não sobe; fica muda.
- **O texto cru do modelo · uma confiança inventada**
  'chuveiro' → 500; um número que não mede nada.

Princípios que levamos

Seis frases para o TP4 — e para qualquer modelo numa aplicação.

- **Quem escolhe, mede**
  no seu dado, na sua máquina — o cartão é pista; o banco de provas, veredito.
- **Custo é memória e tempo**
  e o tipo dos números muda os dois.
- **Quem pesa, carrega uma vez · pensar na CPU é bloquear**
  o lifespan; a rota def.
- **O modelo mora no disco · o modelo cabe no contrato**
  baixe antes; pontue as categorias.
- **Confiança do modelo não é chance de acerto**
  e quem usa precisa saber.

🎬 O caso, em 3 lições (v13–v14)

Da escolha à rota.

- **O mais baixado era inglês; o 'LLM de verdade', gated ou chutando**
  quem escolhe, mede.
- **10 s por análise; o painel congelado**
  carregar uma vez; a rota def.
- **A API muda; o 'chuveiro'**
  baixar antes; o modelo dentro do contrato.

▸ **A Etapa 8 está completa — e o TP4 também.**